



中山大學
SUN YAT-SEN UNIVERSITY

运算符重载 (上)

中山大学计算机学院



主讲人：郑贵锋



联系方式：

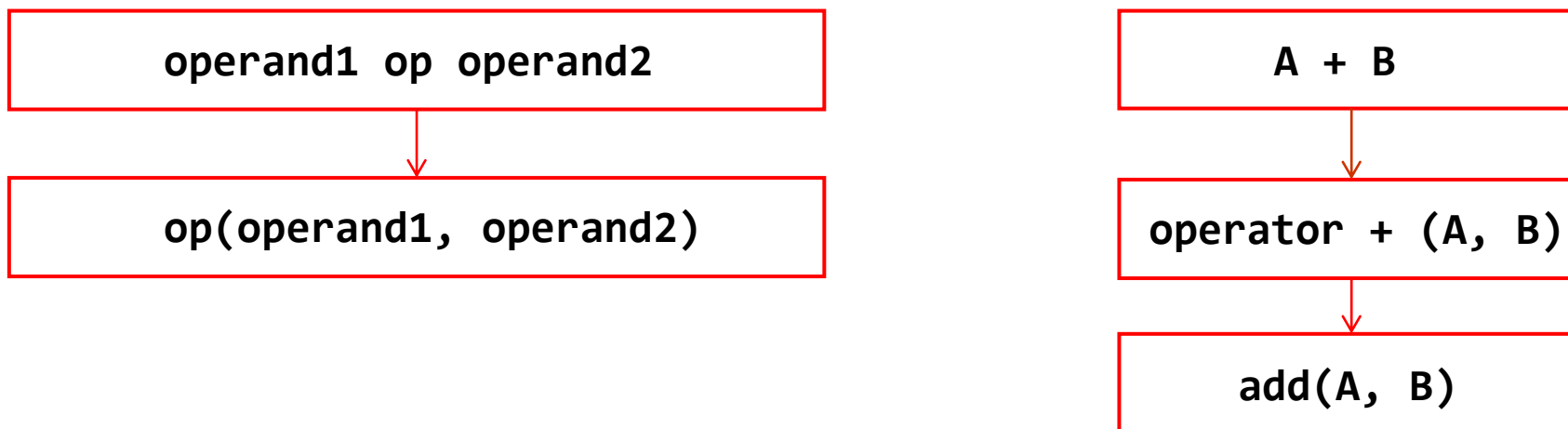
zhenggf@mail.sysu.edu.cn



运算符重载

C++为了增强代码的可读性引入了运算符重载，运算符重载是具有特殊函数名的函数，也具有返回值类型，函数名和参数列表。

重载的运算符可以理解为带有特殊名称的函数，函数名是由关键字 `operator` 和其后要重载的运算符符号构成的。





运算符重载的方法

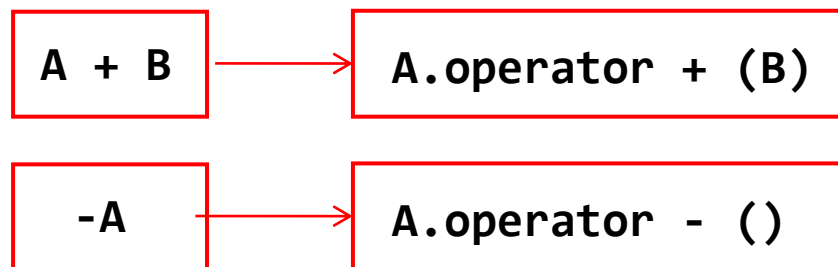
运算符重载的两种方法：

- 类成员函数运算符重载

- `return_type class_name::operator op(operand2) {}`
- 重载二元运算符时，成员运算符函数只需显式传递一个参数（即二元运算符的右操作数），而左操作数则是该类对象本身，通过 `this` 指针隐式传递。
- 重载一元运算符时，成员运算符函数没有参数，操作数是该类对象本身，通过 `this` 指针隐式传递

- 友元函数运算符重载

- `return_type operator op(operand1, operand2) {}`
- 后面会详细说明





可以重载的运算符

	运算符	操作符
双目	算术运算符	+ (加), -(减), *(乘), /(除), %(取模)
	关系运算符	==(等于), != (不等于), < (小于), > (大于), <=(小于等于), >=(大于等于)
	逻辑运算符	(逻辑或), &&(逻辑与), !(逻辑非)
	位运算符	(按位或), & (按位与), ~(按位取反), ^(按位异或), << (左移), >>(右移)
	单目运算符	+ (正), -(负), *(指针), &(取地址)
	自增自减运算符	++(自增), --(自减)
	赋值运算符	=, +=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=
	其他运算符	()(函数调用), -(成员访问), (逗号), [] (下标)
(超纲)	空间申请与释放	new, delete, new[], delete[]



不可以重载的运算符

:: (作用域解析)

. (成员访问)

.* (通过成员指针的成员访问)

?: (三元条件)

sizeof: 运算符 (包括除 **new**, **delete** 外的关键字运算符, 如 **alignof**, **typeid** 等等)

#: 预处理符号

其他限制:

- 不能创建新运算符, 例如 ******、**<>** 或 **&|**。
- 运算符 **&&** 与 **||** 的重载失去短路求值。
- 重载的运算符 **->** 必须要么返回裸指针, 要么 (按引用或值) 返回同样重载了运算符 **->** 的对象。
- 不可能更改运算符的优先级、结合方向或操作数的数量。



双目，单目运算符 - 成员函数重载例子

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    Integer operator + (const Integer& Int) {  
        return Integer(x+Int.x);  
    }  
    Integer operator - (const Integer& Int) {  
        return Integer(x-Int.x);  
    }  
    Integer operator - ()    {  
        return Integer(-x);  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};
```

```
int main() {  
    Integer a=3, b=4;  
    a.print();  
    b.print();  
    Integer c=a+b; // same as a.operator+(b)  
    c.print();  
    Integer d=a-b;  
    d.print();  
    Integer e=-a;  
    e.print();  
    return 0;  
}
```

输出结果:

3
4
7
-1
-3

注意（参见例子中问题）：

第一（左）操作数：必须是 *this;

第二（右）操作数：类型任意，注意区分 T, const T&, T&, T&&, T*, const T* 的差别;

返回值：可以是任意类型，一般是左操作数类型的临时对象T或左操作数自身的引用T&。



注意事项（一）：自增运算符

重载自增运算符时需注意：

- 若为前缀自增运算符，直接重载（以++举例）：
 - `return_type class_name::operator ++()` {}
- 若为后缀自增运算符，该函数有一个 `int` 类型的虚拟形参，这个形参在函数的主体中是不会被使用的，这只是一个约定，它告诉编译器递增运算符正在**后缀模式**下被重载：
 - `return_type class_name::operator ++(int)` {}

运算符名	语法	可重载	原型示例（对于类 <code>class T</code> ）	
			类内定义	类外定义
前自增	<code>++a</code>	是	<code>T& T::operator++();</code>	<code>T& operator++(T& a);</code>
前自减	<code>--a</code>	是	<code>T& T::operator--();</code>	<code>T& operator--(T& a);</code>
后自增	<code>a++</code>	是	<code>T T::operator++(int);</code>	<code>T operator++(T& a, int);</code>
后自减	<code>a--</code>	是	<code>T T::operator--(int);</code>	<code>T operator--(T& a, int);</code>
注解 • 内建运算符的前置版本返回引用而后置版本返回值，典型的用户定义重载也遵循该模式，从而使用户定义运算符可以与内建版本相同的方式使用。然而，用户定义重载中，能以任何类型为返回类型（包括 <code>void</code>）。 • <code>int</code> 形参是用于区别运算符的前置和后置版本的空设形参。调用用户定义的运算符时，该形参中传递的值始终为零，但通过函数调用记法调用这个运算符时可以更改它（例如 <code>a.operator++(2)</code> 或 <code>operator++(a, 2)</code>）。				



注意事项（一）：自增运算符

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x) {  
    }  
    Integer& operator ++ () {  
        cerr<<"prefix is invoked"<<endl;  
        ++x;  
        return *this;  
    }  
    Integer operator ++ (int) {  
        cerr<<"suffix is invoked"<<endl;  
        return Integer(x++);  
    }  
    void print() {  
        cout<<x<<endl;  
    }  
};
```

```
int main() {  
    Integer a=3;  
    a.print();  
    Integer b=++a;  
    a.print();  
    b.print();  
    Integer c=a++;  
    a.print();  
    c.print();  
    return 0;  
}
```

输出结果:

3

prefix is invoked

4

4

suffix is invoked

5

4



注意事项（二）：赋值运算符

C++允许重载赋值运算符：

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    Integer& operator = (const Integer& Int)    {  
        x=Int.x;  
        cout<<"function is invoked"<<endl;  
        return *this;  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};  
  
int main() {  
    Integer a=3, b;  
    a.print();  
    b=a;  
    b.print();  
    return 0;  
}
```

输出结果：

3

function is invoked

3



注意事项（二）：赋值运算符

运算符名	语法	可重载	原型示例（对于 <code>class T</code> ）	
			类内定义	类外定义
简单赋值	<code>a = b</code>	是	<code>T& T::operator =(const T2& b);</code>	N/A
加法赋值	<code>a += b</code>	是	<code>T& T::operator +=(const T2& b);</code>	<code>T& operator +=(T& a, const T2& b);</code>
减法赋值	<code>a -= b</code>	是	<code>T& T::operator -=(const T2& b);</code>	<code>T& operator -=(T& a, const T2& b);</code>
乘法赋值	<code>a *= b</code>	是	<code>T& T::operator *=(const T2& b);</code>	<code>T& operator *=(T& a, const T2& b);</code>
除法赋值	<code>a /= b</code>	是	<code>T& T::operator /=(const T2& b);</code>	<code>T& operator /=(T& a, const T2& b);</code>
取模赋值	<code>a %= b</code>	是	<code>T& T::operator %=(const T2& b);</code>	<code>T& operator %=(T& a, const T2& b);</code>
逐位与赋值	<code>a &= b</code>	是	<code>T& T::operator &=(const T2& b);</code>	<code>T& operator &=(T& a, const T2& b);</code>
逐位或赋值	<code>a = b</code>	是	<code>T& T::operator =(const T2& b);</code>	<code>T& operator =(T& a, const T2& b);</code>
逐位异或赋值	<code>a ^= b</code>	是	<code>T& T::operator ^=(const T2& b);</code>	<code>T& operator ^=(T& a, const T2& b);</code>
逐位左移赋值	<code>a <<= b</code>	是	<code>T& T::operator <<=(const T2& b);</code>	<code>T& operator <<=(T& a, const T2& b);</code>
逐位右移赋值	<code>a >>= b</code>	是	<code>T& T::operator >>=(const T2& b);</code>	<code>T& operator >>=(T& a, const T2& b);</code>
注意 - 所有内建赋值运算符都返回 <code>*this</code>，而大多数用户定义重载也会返回 <code>*this</code>，从而能以与内建版本相同的方式使用用户定义运算符。然而，用户定义运算符重载中，返回类型可以是任意类型（包括 <code>void</code>）。 - T2 可以是包含 T 在内的任意类型				



注意事项（二）：赋值运算符

但会默认有一个缺省的赋值运算符，每个成员变量会直接拷贝值：

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};  
  
int main() {  
    Integer a=3, b;  
    a.print();  
    b=a; // default operator =  
    b.print();  
    return 0;  
}
```

输出结果：

3
3



注意事项（二）：赋值运算符

所以当成员变量包含指针类型的时候，需要注意浅拷贝和深拷贝的区别：

```
class IntArray {
    int *a, n;
public:
    IntArray(int n=1):n(n) {
        a=new int[n];
    }
    ~IntArray() {
        delete[] a;
    }
    int& operator [] (const int& i)
        assert(0<=i&&i<n);
        return a[i];
    }
    void print() {
        for (int i=0;i<n;++i)
            cout<<a[i]<<" ";
        cout<<endl;
    }
};
```

```
int main() {
    IntArray a(4), b;
    for (int i=0;i<4;++i) a[i]=i;
    a.print();
    b=a; // default operator = is ok, but not recommended
    b.print();
    for (int i=0;i<4;++i) a[i]=-i;
    a.print();
    b.print(); // would be same with a
    a.~IntArray();
    b.print(); // undefined behavior, may cause segmentation fault!
    return 0;
}
```

Segmentation fault!!!

1.内存泄漏

2.直接调用析构，会导致多次析构

3.b, a先后析构，段错误

输出结果：

0 1 2 3

0 1 2 3

0 -1 -2 -3

0 -1 -2 -3

2008160928 0 2008154448 0



注意事项（二）：赋值运算符

所以当成员变量包含指针类型的时候，需要注意浅拷贝和深拷贝的区别：

```
class IntArray {
    int *a, n;
public:
    IntArray(int n=1):n(n) {
        a=new int[n];
    }
    ~IntArray() {
        delete[] a;
    }
    int& operator [] (const int& i) {
        assert(0<=i&&i<n);
        return a[i];
    }
    IntArray& operator = (const IntArray& A) {
        delete[] a;
        n=A.n;
        a=new int[n];
        memcpy(a,A.a,sizeof(int)*n);
        return *this;
    }
    void print() {
        for (int i=0;i<n;++i)
            cout<<a[i]<<" ";
        cout<<endl;
    }
};
```

```
int main() {
    IntArray a(4), b;
    for (int i=0;i<4;++i) a[i]=i;
    a.print();
    b=a; // deep copy is good
    b.print();
    for (int i=0;i<4;++i) a[i]=-i;
    a.print();
    b.print(); // would be different from a
    a.~IntArray();
    b.print(); // nothing happens
    return 0;
}
```

输出结果：

0 1 2 3

0 1 2 3

0 -1 -2 -3

0 1 2 3

0 1 2 3



注意事项（三）：移位运算符（类外定义）

在我们初学C++的时候，可能会有这样一个疑问：

- 明明 <<, >> 是数字的移位运算符，为什么他却可以用来读入cin>>x和输出cout<<x呢

现在我们知道了，其实这个就是运算符<<和>>的重载。

同理，我们也可以对自己定义的类做重载：

要点：

- 第一操作数不是 *this，只能类外定义
- 返回对象自身的引用

```
struct Integer {  
    int x;  
};  
  
istream& operator >> (istream& is, Integer& Int) {  
    is>>Int.x;  
    return is;  
}  
  
ostream& operator << (ostream& os, const Integer& Int) {  
    os<<Int.x;  
    return os;  
}  
  
int main() {  
    Integer x;  
    cin>>x;  
    cout<<x<<endl;  
    return 0;  
}
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



主讲人：郑贵锋



中山大学MOOC课程组